CPE 486/586: Deep Learning for Engineering Applications

12 Neural Ordinary Differential Equations

Spring 2026

Rahul Bhadani

Outline

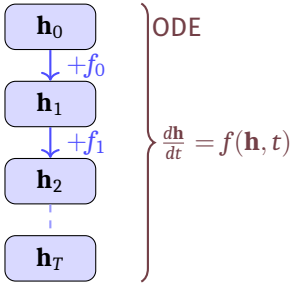
Neural ODEs in the Context of Discrete Dynamical Systems

Deep Networks as Discrete Dynamical Systems

- Modern deep networks (ResNets, DenseNets) stack many layers
- Each layer transforms a hidden state:

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t)$$

- This is **Euler's method** applied to an ODE!
- As layer count $\to \infty$, the discrete steps become a continuous flow

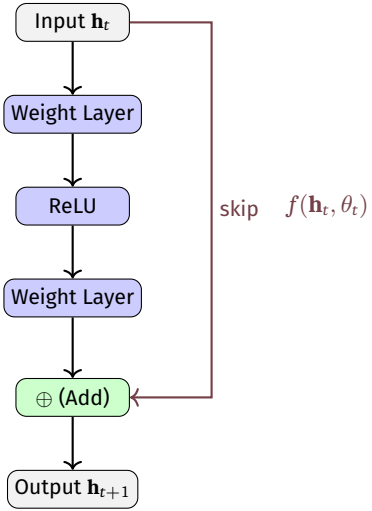


Residual Networks (ResNets)

ResNet Building Block

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t), \quad t = 0, 1, \ldots, T-1$$

- $\mathbf{h}_t \in \mathbb{R}^d$: hidden state at layer t
- f: neural network (residual function)
- Skip connection ensures gradient flow
- Enables very deep networks (100+ layers)



Connection to Euler Method

Euler's method for $\frac{d\mathbf{h}}{dt} = f(\mathbf{h}, t)$ with step $\Delta t = 1$:

$$\mathbf{h}(t + \Delta t) \approx \mathbf{h}(t) + \Delta t \cdot f(\mathbf{h}(t), t)$$

From Discrete to Continuous Depth

As number of layers $T \to \infty$ and step size $\Delta t \to 0$, the residual update

$$\mathbf{h}_{t+1} = \mathbf{h}_t + \Delta t f(\mathbf{h}_t,\, t,\, \theta)$$

converges to the **continuous ODE**:

$$\boxed{\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t),\, t,\, \theta)}$$

Discrete: ResNet

$$\mathbf{h}_T = \mathbf{h}_0 + \sum_{t=0}^{T-1} f(\mathbf{h}_t, \theta_t)$$

Continuous: Neural ODE

$$\mathbf{h}(T) = \mathbf{h}(0) + \int_0^T f(\mathbf{h}(t), t, \theta)\, dt$$

The parameters θ are **shared across all depths,** unlike ResNets with per-layer weights. This gives **constant parameter count** regardless of integration depth.

Ordinary Differential Equations (ODEs)

Initial Value Problem (IVP)

Given dynamics f and initial condition $\mathbf{h}(t_0) = \mathbf{h}_0$, find $\mathbf{h}(t)$:

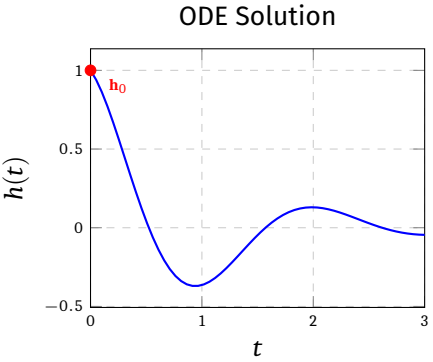
$$\frac{d\mathbf{h}}{dt} = f(\mathbf{h}(t), t), \quad \mathbf{h}(t_0) = \mathbf{h}_0$$

Solution (via Picard–Lindelöf):

$$\mathbf{h}(t) = \mathbf{h}_0 + \int_{t_0}^{t} f(\mathbf{h}(s), s)\, ds$$

Existence & Uniqueness

If f is **Lipschitz continuous** in $\mathbf{h}$, a unique solution exists on $[t_0, T]$.



Numerical ODE Solvers

Euler Method (1st Order)

$$\mathbf{h}_{t+h} \approx \mathbf{h}_t + h f(\mathbf{h}_t, t)$$

Simple but low accuracy; $\mathcal{O}(h)$ error per step.

Runge-Kutta 4 (RK4)

$$\begin{aligned} k_1 &= f(\mathbf{h}_t,\, t) \\ k_2 &= f\left(\mathbf{h}_t + \tfrac{h}{2}k_1,\, t + \tfrac{h}{2}\right) \\ k_3 &= f\left(\mathbf{h}_t + \tfrac{h}{2}k_2,\, t + \tfrac{h}{2}\right) \\ k_4 &= f(\mathbf{h}_t + h\,k_3,\, t + h) \\ \mathbf{h}_{t+h} &= \mathbf{h}_t + \tfrac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

$\mathcal{O}(h^4)$ error per step.

Adaptive Solvers (Dopri5)

- Embed two RK schemes of different orders
- Estimate local truncation error
- **Increase** step size when error is small
- **Decrease** step size when error is large
- Control solution to a user-specified tolerance: `rtol`, `atol`

Neural ODE dynamics can vary rapidly in some regions; adaptive solvers allocate computation where needed, yielding efficiency and accuracy.

Neural ODEs: The Core Idea

Definition

A **Neural ODE** parametrizes the derivative of a hidden state with a neural network f_θ:

$$\frac{d\mathbf{h}(t)}{dt} = f_\theta(\mathbf{h}(t),\, t)\,, \qquad \mathbf{h}(t_0) = \mathbf{h}_0$$

The output at time t_1 is computed by **numerically integrating** the ODE:

$$\mathbf{h}(t_1) = \mathbf{h}(t_0) + \int_{t_0}^{t_1} f_\theta(\mathbf{h}(t), t)\, dt \;=\; \texttt{ODESolve}(f_\theta,\, \mathbf{h}_0,\, t_0,\, t_1)$$

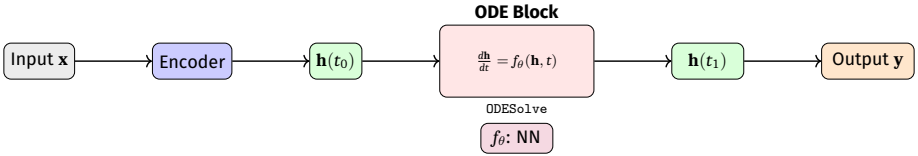
Forward Pass

1. Set initial state $\mathbf{h}(t_0)$ (e.g., encoder output)
2. Call ODE solver with f_θ
3. Obtain $\mathbf{h}(t_1)$ as the model output
4. Apply output layer / loss

Backward Pass

1. Do **NOT** backprop through solver steps
2. Solve a **reverse-time ODE** (adjoint)
3. Compute gradients $\nabla_\theta \mathcal{L}$ efficiently
4. Memory cost: $\mathcal{O}(1)$ in depth

Neural ODE: Architecture View



Continuous Depth

The "depth" of the network is the integration interval $[t_0, t_1]$. More complex inputs naturally require more solver steps (adaptive depth).

Parameter Efficiency

f_θ is **shared** across all integration steps. Total parameters determined by the architecture of f_θ, *not* by integration depth.

Neural ODE for Classification

1. **Downsampling conv layers**: extract features from input image $\mathbf{x}$
2. **ODE block**: continuous feature evolution

$$\frac{d\mathbf{h}}{dt} = f_\theta(\mathbf{h}(t), t)$$

3. **Fully connected layer**: map $\mathbf{h}(t_1) \rightarrow$ class logits

Why the ODE block is invertible?

Given $\mathbf{h}(t_1)$, integrate the ODE **backwards** in time from t_1 to t_0:

$$\mathbf{h}(t_0) = \mathbf{h}(t_1) + \int_{t_1}^{t_0} f_\theta(\mathbf{h}(t), t)\, dt$$

This gives exact **free invertibility**, an interesting property for normalizing flows!

The Adjoint Sensitivity Method: Overview

The Problem

To get trained model with parameters θ we need $\frac{d\mathcal{L}}{d\theta}$. Naively backpropping through the ODE solver requires storing all intermediate states $\Rightarrow \mathcal{O}(N_{\text{steps}})$ memory.

Adjoint State

Define the **adjoint** $\mathbf{a}(t) = \frac{\partial \mathcal{L}}{\partial \mathbf{h}(t)}$.

- $\mathbf{a}(t_1) = \frac{\partial \mathcal{L}}{\partial \mathbf{h}(t_1)}$ (from loss at output)
- $\mathbf{a}(t)$ evolves **backwards** via its own ODE:

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^{\mathsf{T}} \frac{\partial f_\theta(\mathbf{h}(t), t, \theta)}{\partial \mathbf{h}(t)}$$

Memory Advantage

The adjoint ODE is solved **backwards in time**, recomputing $\mathbf{h}(t)$ on the fly. No need to store forward trajectory. Memory cost: $\mathcal{O}(1)$ (constant w.r.t. solver steps)!

Adjoint Method: Gradient Computation

Augmented ODE (Backward in Time)

Solve the following augmented system from t_1 to t_0:

$$\frac{d}{dt}\begin{bmatrix}\mathbf{h}(t)\\ \mathbf{a}(t)\\ \frac{d\mathcal{L}}{d\theta}\end{bmatrix} = \begin{bmatrix} f_\theta(\mathbf{h},t)\\ -\mathbf{a}^\top \frac{\partial f_\theta}{\partial \mathbf{h}}\\ -\mathbf{a}^\top \frac{\partial f_\theta}{\partial \theta}\end{bmatrix}$$

with initial conditions at t_1:

$$\mathbf{h}(t_1), \quad \mathbf{a}(t_1) = \frac{\partial \mathcal{L}}{\partial \mathbf{h}(t_1)}, \quad \left.\frac{d\mathcal{L}}{d\theta}\right|_{t_1} = \mathbf{0}$$

After integrating to t_0:

- $\mathbf{a}(t_0) = \frac{\partial \mathcal{L}}{\partial \mathbf{h}(t_0)}$ (gradient w.r.t. initial state)
- $\frac{d\mathcal{L}}{d\theta}$ accumulated across all time (gradient w.r.t. parameters)

Computation Summary

Quantity	Cost
Forward solve	1 ODE call
Backward solve	1 ODE call
Memory	$\mathcal{O}(1)$
Jacobian-vecs	via autograd

Adjoint Method: Algorithm

Algorithm 1: Neural ODE Training with Adjoint Sensitivity

Input: Parameters θ, initial state $\mathbf{h}_0$, loss $\mathcal{L}$, times $t_0 < t_1$
Output: Gradients $\frac{d\mathcal{L}}{d\theta}$, $\frac{d\mathcal{L}}{d\mathbf{h}_0}$

Forward pass:
$\mathbf{h}(t_1) \leftarrow \texttt{ODESolve}(f_\theta, \mathbf{h}_0, t_0, t_1)$;
Compute $\mathcal{L}(\mathbf{h}(t_1))$;

Backward pass (Adjoint):
Initialize $\mathbf{a}(t_1) = \frac{\partial \mathcal{L}}{\partial \mathbf{h}(t_1)}$;;
$\mathbf{g}_\theta = \mathbf{0}$;
Solve augmented ODE from t_1 to t_0: $[\mathbf{h}(t_0),\ \mathbf{a}(t_0),\ \mathbf{g}_\theta] \leftarrow \texttt{ODESolve}(\text{aug}, [\mathbf{h}_1, \mathbf{a}_1, \mathbf{0}], t_1, t_0)$;
return $\frac{d\mathcal{L}}{d\mathbf{h}_0} = \mathbf{a}(t_0)$;;
;
$\frac{d\mathcal{L}}{d\theta} = \mathbf{g}_\theta$;

The backward augmented ODE is solved with the *same* ODE solver as the forward pass, the solver is a black box, gradients flow through the **adjoint ODE**, not through solver steps.

Normalizing Flows: Background

Change of Variables

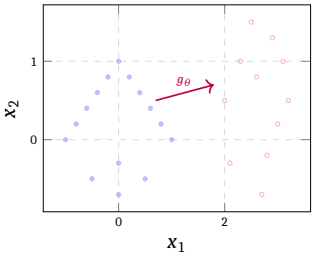
A normalizing flow maps a simple base distribution $p_z(\mathbf{z})$ (e.g., Gaussian) to a complex target $p_x(\mathbf{x})$ via an invertible map $g : \mathbf{z} \mapsto \mathbf{x}$:

$$\log p_x(\mathbf{x}) = \log p_z(\mathbf{z}) - \log \left| \det \frac{\partial g}{\partial \mathbf{z}} \right|$$

Discrete NF Challenges

- Must design **special invertible architectures**
- Jacobian determinant computation: $\mathcal{O}(d^3)$ in general
- Coupling layers restrict architecture freedom

Flow: Simple → Complex



Continuous Normalizing Flows (CNF)

Instantaneous Change of Variables

For a continuous transformation $\mathbf{z}(t)$ governed by an ODE, the log-density evolves as:

$$\frac{d \log p(\mathbf{z}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial f_\theta}{\partial \mathbf{z}(t)}\right)$$

This is the **instantaneous change of variables** formula (Liouville's theorem).

CNF Log-Likelihood

$$\log p(\mathbf{x}) = \log p(\mathbf{z}(t_0)) - \int_{t_0}^{t_1} \operatorname{tr}\left(\frac{\partial f_\theta}{\partial \mathbf{z}}\right) dt$$

- $p(\mathbf{z}(t_0))$: base density (Gaussian)
- Trace computable in $\mathcal{O}(d)$ via Hutchinson's estimator

Advantages over Discrete NF

- **Free-form** f_θ, no special invertibility constraints
- Only **trace** (not full Jacobian) needed
- Can model highly complex distributions
- Generation: integrate ODE forward; inference: backward

Hutchinson's Trace Estimator

Problem

Computing $\mathrm{tr}(J_f)$ where $J_f = \frac{\partial f_\theta}{\partial \mathbf{z}} \in \mathbb{R}^{d \times d}$ costs $\mathcal{O}(d^2)$, expensive for large d!

Hutchinson Estimator

For any random vector $\varepsilon \in \mathbb{R}^d$ with $\mathbb{E}[\varepsilon] = 0$, $\mathbb{E}[\varepsilon \varepsilon^\mathsf{T}] = I$:

$$\mathrm{tr}(J_f) = \mathbb{E}_\varepsilon \left[\varepsilon^\mathsf{T} J_f \, \varepsilon \right]$$

- $\varepsilon^\mathsf{T} J_f$ is a **vector-Jacobian product**: computable via one backward pass through f_θ, costing $\mathcal{O}(d)$
- Use Rademacher noise: $\varepsilon_i \in \{-1, +1\}$ with equal probability
- Average over K samples for low-variance estimate

Training CNFs requires only $\mathcal{O}(d)$ operations per step, making high-dimensional density estimation tractable.

Latent Neural ODEs for Time Series

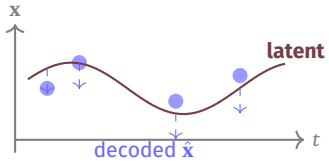
Motivation: Irregular Time Series

Standard RNNs process observations at fixed, regular intervals. Real-world data is:

- **Irregularly sampled**: medical records, sensor readings
- **Partially observed**: missing measurements
- **Multi-rate**: different sensors at different frequencies

Latent ODE Model

1. **Encoder** (ODE-RNN): infer initial latent state $\mathbf{z}(t_0)$ from observations
2. **Latent dynamics**: $\frac{d\mathbf{z}}{dt} = f_\theta(\mathbf{z}(t))$ (no explicit t-dependence)
3. **Decoder**: map latent $\mathbf{z}(t_k) \to \hat{\mathbf{x}}(t_k)$ at any time t_k



ODE-RNN Encoder

ODE-RNN

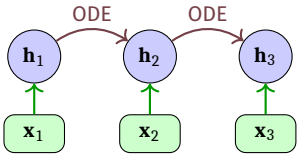
Combine an RNN (for observation updates) with an ODE (for dynamics between observations):

- At observation time t_k: **RNN update**

$$\mathbf{h}(t_k^+) = \text{RNNCell}\Big(\mathbf{h}(t_k^-),\, \mathbf{x}(t_k)\Big)$$

- Between observations: **ODE evolution**

$$\mathbf{h}(t_{k+1}^-) = \texttt{ODESolve}(f_\theta, \mathbf{h}(t_k^+), t_k, t_{k+1})$$



Advantage

Handles **arbitrarily irregular** time gaps between observations naturally.

Latent ODE: Generative Model

Variational Formulation

Model observations as:

$$\mathbf{z}(t_0) \sim p(\mathbf{z}_0), \quad \mathbf{z}(t) = \mathbf{z}(t_0) + \int_{t_0}^{t} f_\theta(\mathbf{z}(s))\, ds, \quad \mathbf{x}(t_k) \sim p(\mathbf{x} \mid \mathbf{z}(t_k))$$

Train with **ELBO** (Evidence Lower Bound):

$$\mathcal{L} = \mathbb{E}_{q(\mathbf{z}_0 \mid \{\mathbf{x}(t_k)\})} \left[\sum_k \log p(\mathbf{x}(t_k) \mid \mathbf{z}(t_k)) \right] - \mathrm{KL}[q(\mathbf{z}_0 \mid \cdot) \,\|\, p(\mathbf{z}_0)]$$

Capabilities

- Interpolate at **any time** (not just observed)
- Extrapolate into future
- Handle **missing data** gracefully
- Uncertainty quantification

Applications

- Clinical time series (ICU data)
- Physical system identification
- Financial time series
- Motion capture / trajectory modeling

Implementation with `torchdiffeq`

`torchdiffeq` Library (Chen et al.)

- PyTorch-based library for differentiable ODE solving
- Supports: `euler`, `rk4`, `dopri5` (Dormand–Prince), `adams`
- Automatic adjoint-based gradients

```python
from torchdiffeq import odeint, odeint_adjoint
import torch, torch.nn as nn

class ODEFunc(nn.Module):
    """Parametrize dh/dt = f_theta(h, t)"""
    def __init__(self, hidden_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
        )
    def forward(self, t, h):   # Note: signature must be (t, h)
        return self.net(h)

func = ODEFunc(hidden_dim=64)
h0   = torch.randn(16, 64)          # batch_size x hidden_dim
t    = torch.tensor([0.0, 1.0])     # integration interval
# odeint_adjoint uses O(1) memory backprop
h1   = odeint_adjoint(func, h0, t, method='dopri5')[-1]
```

Complete Neural ODE Classifier

```
class NeuralODEClassifier(nn.Module):
    def __init__(self, in_dim=784, hidden=64, n_classes=10):
        super().__init__()
        self.encoder  = nn.Linear(in_dim, hidden)
        self.ode_func = ODEFunc(hidden)
        self.decoder  = nn.Linear(hidden, n_classes)
        self.t        = torch.tensor([0.0, 1.0])
    def forward(self, x):
        h0 = self.encoder(x)                          # (B, hidden)
        # Solve ODE: O(1) memory, adaptive steps
        hT = odeint_adjoint(self.ode_func, h0,
                            self.t, method='dopri5')[-1]
        return self.decoder(hT)                       # logits
# Training loop (standard)
model     = NeuralODEClassifier()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()
for x_batch, y_batch in dataloader:
    logits = model(x_batch.view(x_batch.size(0), -1))
    loss   = criterion(logits, y_batch)
    optimizer.zero_grad()
    loss.backward()      # adjoint handles backprop
    optimizer.step()
```

ODE Solver Options in `torchdiffeq`

Available Solvers

Method	Order / Type
`euler`	1st, fixed-step
`midpoint`	2nd, fixed-step
`rk4`	4th, fixed-step
`dopri5`	4/5th, adaptive
`dopri8`	8th, adaptive
`adams`	multi-step, adaptive
`bosh3`	2/3rd, adaptive

Use `dopri5` for training (accuracy), `rk4` with fixed step for fast inference.

```python
# Tight tolerances for high accuracy
h = odeint_adjoint(
    func, h0, t,
    method='dopri5',
    rtol=1e-5,
    atol=1e-6
)

# Faster inference: fixed-step
h = odeint(
    func, h0, t,
    method='rk4',
    options={'step_size': 0.1}
)
```

Advantages of Neural ODEs

Core Strengths

- **Memory Efficiency**
  Adjoint backprop: $\mathcal{O}(1)$ memory vs. $\mathcal{O}(L)$ for ResNet
- **Adaptive Computation**
  ODE solver uses more steps for complex regions; fewer steps where dynamics are simple
- **Continuous Dynamics**
  Outputs defined at *any* time $t \in [t_0, t_1]$, not just integer layers
- **Invertibility**
  Free inversion via backward ODE integration — crucial for generative models

Application-Specific Gains

- **Irregular Time Series**
  Handles variable time gaps naturally
- **Continuous Normalizing Flows**
  Exact log-likelihood with free-form architecture
- **Parameter Efficiency**
  Weights shared across all "depths" (integration steps)
- **Physical Inductive Bias**
  Natural fit for systems governed by differential equations (mechanics, chemistry, biology)

Limitations and Challenges

Computational Challenges

- **Slow training**: ODE solving is iterative; each gradient step requires forward + backward ODE integration
- **Stiff ODEs**: dynamics with very different timescales require tiny step sizes
- **Adjoint inaccuracy**: numerical errors in backward ODE accumulate; tight tolerances needed
- **Depth-accuracy tradeoff**: coarser tolerance $\Rightarrow$ fewer steps but less accurate

Modeling Limitations

- **No crossing trajectories**: homeomorphism constraint — the ODE flow is a diffeomorphism; cannot fold or cross in latent space
- **Topology**: cannot change topology of data manifold (e.g., cannot map one cluster to two disjoint clusters without augmentation)
- **Expressiveness**: augment state space to overcome (Augmented Neural ODEs)
- **Hyperparameter sensitivity**: solver tolerance, integration range need tuning

Augmented Neural ODEs

Motivation: Overcoming Topology Limitations

Recall: an ODE flow $\phi : \mathbb{R}^d \to \mathbb{R}^d$ is a **homeomorphism** — cannot change topology. For example, points on opposite sides of a boundary cannot swap during integration (no crossing).

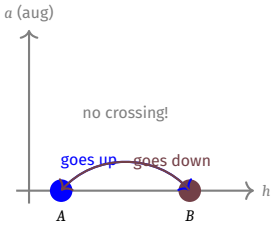
Augmented Neural ODE (ANODE)

Augment the state with p additional dimensions initialized to zero:

$$\tilde{\mathbf{h}}(t_0) = \begin{bmatrix} \mathbf{h}(t_0) \\ \mathbf{0}_p \end{bmatrix} \in \mathbb{R}^{d+p}$$

Solve the ODE in the augmented space $\mathbb{R}^{d+p}$, then read off the first d dimensions.

- Extra dimensions provide "lifting" room to route around topological barriers
- Small p (e.g., $p = 1$) often sufficient in practice



Extensions: Neural CDEs and SDEs

Neural Controlled Differential Equations (CDEs)

$$\mathbf{z}(T) = \mathbf{z}(t_0) + \int_{t_0}^{T} f_\theta(\mathbf{z}(t))\, d\mathbf{X}(t)$$

- Generalizes Neural ODEs with a **control signal** $\mathbf{X}(t)$ (the data path)
- Rigorously handles **online** time-series processing
- Handles irregular observations via the **Stieltjes integral**
- Theoretically grounded: classical CDE theory (Lyons, 1998)

Neural Stochastic DEs (SDEs)

$$d\mathbf{z}(t) = f_\theta(\mathbf{z}(t), t)\, dt + g_\theta(\mathbf{z}(t), t)\, d\mathbf{W}(t)$$

- $\mathbf{W}(t)$: Wiener process (Brownian motion)
- g_θ: diffusion / noise network
- Models **stochasticity** in dynamics
- Equivalent to infinite-width Bayesian neural network in some limits
- Score-based generative models (diffusion models) are a special case!

Connection to Score-Based Generative Models

Probability Flow ODE

Score-based diffusion models define a forward SDE that gradually adds noise to data. There exists an equivalent **deterministic ODE** (probability flow ODE) with the same marginal densities:

$$d\mathbf{x} = \left[f(\mathbf{x}, t) - \frac{1}{2} g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt$$

- $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$: **score function**, learned by a neural network
- This is a Neural ODE with a score-network as the dynamics!
- Enables **exact log-likelihood** computation (CNF connection)
- Allows **controllable generation** and inversion

Neural ODEs form the theoretical backbone of modern **diffusion-based generative models** (DDPM, Score SDE, Flow Matching).

TLDR

1. **Neural ODEs** replace discrete layer stacks with a continuous ODE: $\frac{d\mathbf{h}}{dt} = f_\theta(\mathbf{h}, t)$
2. **Forward pass**: call a black-box ODE solver (`ODESolve`)
3. **Backward pass**: adjoint sensitivity method — $\mathcal{O}(1)$ memory, numerically stable
4. **CNFs**: exact log-likelihood via instantaneous change of variables + trace estimator
5. **Latent Neural ODEs**: continuous-time generative model for irregular time series
6. **Augmented NODEs**: overcome topology limitations by lifting to higher dimensions
7. **Extensions**: Neural CDEs (online data), Neural SDEs (stochasticity), connect to diffusion models

References

- Chen et al. (2018) — *Neural Ordinary Differential Equations*, NeurIPS
- Grathwohl et al. (2018) — *FFJORD*, ICLR 2019
- Rubanova et al. (2019) — *Latent ODEs*, NeurIPS
- Kidger et al. (2020) — *Neural CDEs*
- Song et al. (2021) — *Score-Based Generative Modeling through SDEs*

Neural ODEs unify **dynamical systems** and **deep learning**: they give continuous-depth models, constant-memory training, adaptive computation, and a principled framework for continuous-time sequence modeling and generative modeling.